



University of Asia Pacific

Department of Computer Science and Engineering

CSE 402 : Operating System Lab

LAB REPORT

Lab Report Number: 04

Experiment : Page Replacement Algorithms – FIFO, LRU, and Optimal

Submitted by:

Name : H. M. Tahsin Sheikh

Student ID : 22201243

Section : E1

Submitted to:

Bidita Sarkar Diba

Lecturer,

Department of Computer Science and Engineering

Date of Submission: 06/05/2026

1. Title of the Experiment

Page Replacement Algorithms – FIFO, LRU, and Optimal

2. Objective

The objectives of this experiment are:

- To understand the working principles of FIFO, LRU, and Optimal page replacement algorithms.
- To simulate page replacement using Python.
- To calculate the number of page faults for each algorithm.
- To compare the performance of different page replacement techniques.

3. Introduction

Page Replacement is a memory management technique used in operating systems when the number of pages exceeds the available frames in memory. When a new page is required, the system must decide which page to remove.

In this experiment, we implement:

FIFO (First In First Out) Algorithm

- The oldest page in memory is replaced first.
- Pages are removed in the order they were loaded.

Advantages:

- Simple to understand and implement
- Low overhead

LRU (Least Recently Used) Algorithm

- The page that has not been used for the longest time is replaced.
- Uses past access patterns to predict future usage.

Advantages:

- Better performance than FIFO
- Reduces page faults

Optimal Page Replacement Algorithm

- Replaces the page that will not be used for the longest time in the future.
- Provides the best possible performance.

Advantages:

- Minimum number of page faults
- Used as a benchmark for comparison

4. Algorithm

Algorithm for FIFO

1. Start
2. Input page reference string
3. Input number of frames
4. Initialize empty frames
5. For each page:
6. If page not in memory → page fault
7. If frames are full → remove oldest page
8. Insert new page
9. Count page faults
10. Stop

Algorithm for LRU

1. Start
2. Input page reference string
3. Input number of frames
4. Initialize empty frames
5. For each page:
6. If page not in memory → page fault
7. Replace least recently used page
8. Update usage order
9. Count page faults
10. Stop

Algorithm for Optimal

1. Start
2. Input page reference string
3. Input number of frames
4. Initialize empty frames
5. For each page:
6. If page not in memory → page fault
7. Replace page that will not be used for longest time
8. Count page faults
9. Stop

5. Source Code

FIFO Algorithm Code –

```
n = int(input("Enter number of processes: "))

process = []
burst = []

for i in range(n):
    process.append("P" + str(i + 1))
    b = int(input(f"Enter Burst Time for P{i+1}: "))
    burst.append(b)

waiting = [0] * n

# Calculate waiting time
for i in range(1, n):
    waiting[i] = waiting[i - 1] + burst[i - 1]

total_waiting = sum(waiting)
avg_waiting = total_waiting / n

print("\nProcess\tBurst\tWaiting")
for i in range(n):
    print(process[i], "\t", burst[i], "\t", waiting[i])

print("\nAverage Waiting Time =", avg_waiting)
```

LRU Algorithm Code –

```
def print_frames(frames, capacity):
    temp = frames.copy()
    while len(temp) < capacity:
        temp.insert(0, "-")
    print("Frames:", temp)
```

```

def lru(pages, capacity):
    frames = []
    recent = {}
    faults = 0

    print("\n--- LRU ---")
    for i, p in enumerate(pages):
        if p in frames:
            print(f"{p} -> Hit", end=" | ")
        else:
            faults += 1
            if len(frames) < capacity:
                frames.append(p)
            else:
                lru_page = min(frames, key=lambda x: recent[x])
                frames.remove(lru_page)
                frames.append(p)
            print(f"{p} -> Miss", end=" | ")

        recent[p] = i
        print_frames(frames, capacity)

    print("Total Page Faults:", faults)

# ♦ Input
pages = list(map(int, input("Enter page reference string: ").split()))
capacity = int(input("Enter number of frames: "))

lru(pages, capacity)

```

Optimal Algorithm Code –

```

def print_frames(frames, capacity):
    temp = frames.copy()
    while len(temp) < capacity:
        temp.insert(0, '-')
    print("Frames:", temp)

def optimal(pages, capacity):
    frames = []
    faults = 0

    print("\n--- Optimal ---")
    for i, p in enumerate(pages):
        if p in frames:
            print(f"{p} -> Hit", end=" | ")
        else:
            faults += 1
            if len(frames) < capacity:
                frames.append(p)

```

```

else:
    future = pages[i+1:]
    farthest = -1
    replace = None

    for f in frames:
        if f not in future:
            replace = f
            break
        else:
            idx = future.index(f)
            if idx > farthest:
                farthest = idx
                replace = f

    frames.remove(replace)
    frames.append(p)

    print(f"{p} -> Miss", end=" | ")

    print_frames(frames, capacity)

print("Total Page Faults:", faults)

# ◆ Input
pages = list(map(int, input("Enter page reference string: ").split()))
capacity = int(input("Enter number of frames: "))

optimal(pages, capacity)

```

6. Input/Output

◆ Sample Input :

```

Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2
Enter number of frames: 3

```

◆ Output for FIFO :

```

main.py
1- def print_frames(frames, capacity):
2-     temp = frames.copy()
3-     while len(temp) < capacity:
4-         temp.insert(0, "-")
5-         print("Frames:", temp)
6-
7-
8- def fifo(pages, capacity):
9-     frames = []
10-    faults = 0
11-
12-    print("\n--- FIFO ---")
13-    for p in pages:
14-        if p in frames:
15-            print(f"{p} -> Hit", end=" | ")
16-        else:
17-            faults += 1
18-            if len(frames) < capacity:
19-                frames.append(p)
20-            else:
21-                frames.pop(0) # remove oldest page
22-                frames.append(p)
23-            print(f"{p} -> Miss", end=" | ")

```

```

Output
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2
Enter number of frames: 3

--- FIFO ---
7 -> Miss | Frames: ['-', '-', 7]
0 -> Miss | Frames: ['-', 7, 0]
1 -> Miss | Frames: [7, 0, 1]
2 -> Miss | Frames: [0, 1, 2]
0 -> Hit | Frames: [0, 1, 2]
3 -> Miss | Frames: [1, 2, 3]
0 -> Miss | Frames: [2, 3, 0]
4 -> Miss | Frames: [3, 0, 4]
2 -> Miss | Frames: [0, 4, 2]
3 -> Miss | Frames: [4, 2, 3]
0 -> Miss | Frames: [2, 3, 0]
3 -> Hit | Frames: [2, 3, 0]
2 -> Hit | Frames: [2, 3, 0]
Total Page Faults: 10

=== Code Execution Successful ===

```

◆ Output for Optimal:

```

main.py
14- if p in frames:
15-     print(f"{p} -> Hit", end=" | ")
16- else:
17-     faults += 1
18-     if len(frames) < capacity:
19-         frames.append(p)
20-     else:
21-         future = pages[i+1:]
22-         farthest = -1
23-         replace = None
24-
25-         for f in frames:
26-             if f not in future:
27-                 replace = f
28-                 break
29-         else:
30-             idx = future.index(f)
31-             if idx > farthest:
32-                 farthest = idx
33-                 replace = f
34-
35-         frames.remove(replace)
36-         frames.append(p)

```

```

Output
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2
Enter number of frames: 3

--- Optimal ---
7 -> Miss | Frames: ['-', '-', 7]
0 -> Miss | Frames: ['-', 7, 0]
1 -> Miss | Frames: [7, 0, 1]
2 -> Miss | Frames: [0, 1, 2]
0 -> Hit | Frames: [0, 1, 2]
3 -> Miss | Frames: [0, 2, 3]
0 -> Hit | Frames: [0, 2, 3]
4 -> Miss | Frames: [2, 3, 4]
2 -> Hit | Frames: [2, 3, 4]
3 -> Hit | Frames: [2, 3, 4]
0 -> Miss | Frames: [2, 3, 0]
3 -> Hit | Frames: [2, 3, 0]
2 -> Hit | Frames: [2, 3, 0]
Total Page Faults: 7

=== Code Execution Successful ===

```

◆ Output for LRU:

```

main.py
1- def print_frames(frames, capacity):
2-     temp = frames.copy()
3-     while len(temp) < capacity:
4-         temp.insert(0, "-")
5-         print("Frames:", temp)
6-
7-
8- def lru(pages, capacity):
9-     frames = []
10-    recent = {}
11-    faults = 0
12-
13-    print("\n--- LRU ---")
14-    for i, p in enumerate(pages):
15-        if p in frames:
16-            print(f"{p} -> Hit", end=" | ")
17-        else:
18-            faults += 1
19-            if len(frames) < capacity:
20-                frames.append(p)
21-            else:
22-                lru_page = min(frames, key=lambda x: recent[x])
23-                frames.remove(lru_page)

```

```

Output
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2
Enter number of frames: 3

--- LRU ---
7 -> Miss | Frames: ['-', '-', 7]
0 -> Miss | Frames: ['-', 7, 0]
1 -> Miss | Frames: [7, 0, 1]
2 -> Miss | Frames: [0, 1, 2]
0 -> Hit | Frames: [0, 1, 2]
3 -> Miss | Frames: [0, 2, 3]
0 -> Hit | Frames: [0, 2, 3]
4 -> Miss | Frames: [0, 3, 4]
2 -> Miss | Frames: [0, 4, 2]
3 -> Miss | Frames: [4, 2, 3]
0 -> Miss | Frames: [2, 3, 0]
3 -> Hit | Frames: [2, 3, 0]
2 -> Hit | Frames: [2, 3, 0]
Total Page Faults: 9

=== Code Execution Successful ===

```

7. Conclusion

From this experiment, we successfully implemented FIFO, LRU, and Optimal page replacement algorithms.

We observed that:

- FIFO is simple but may lead to more page faults.
- LRU performs better by considering recent usage.
- Optimal gives the least number of page faults but is not practical in real systems.
- Among these, Optimal is best in theory, while LRU is widely used in practice.

These page replacement algorithms help improve memory utilization, reduce page faults, and enhance overall system performance in operating systems.